



Shared CPU-FPGA Filesystem Access on fast NVMe Devices

Master's Thesis

Author: Bc. VLADISLAV VÁLEK
Advisor: Prof. Dr. DIRK KOCH
Opponent: Prof. Dr. HOLGER FRÖNING

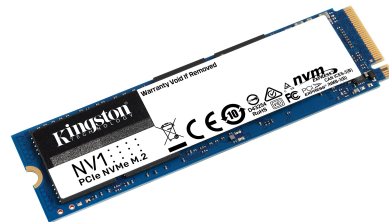
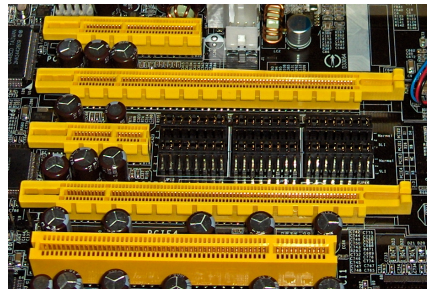
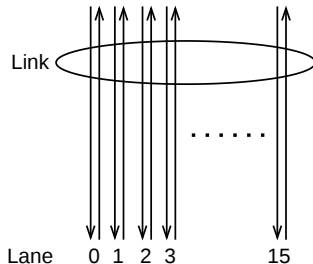
Heidelberg, 30-March-2026

- Motivation and goals
- PCI Express and NVMe essentials
- DMA Iuventus architecture
- Shared filesystem access
- Evaluation highlights
- Conclusion

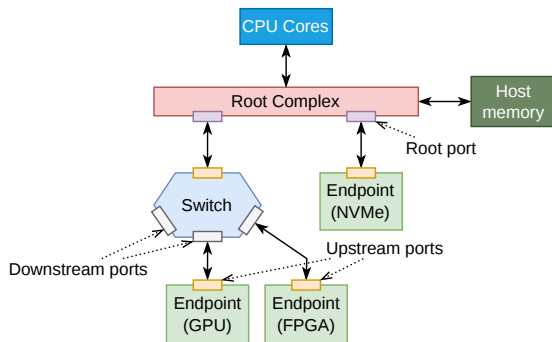
- Direct access to persistent data from accelerator
- Conventional CPU-based copies introduce an overhead
- Host memory used for staging (double copying)
- With peer-to-peer transfers, the NVMe can be directly accessed
- Linux user friendly (standardized filesystem)

Goal of this work: enable shared CPU-FPGA filesystem access to NVMe while preserving a standard Linux workflow.

- Switched point-to-point interconnect
- Memory-mapped communication
- Support on FPGAs



- Tree topology
- Root Complex, Switches, Endpoints
- Address-based routing (for our case :)
- Host bypass possible if platform permits P2P
- Transaction Layer Packet (TLP)
 - Memory Write request
 - Memory Read request
 - Completion



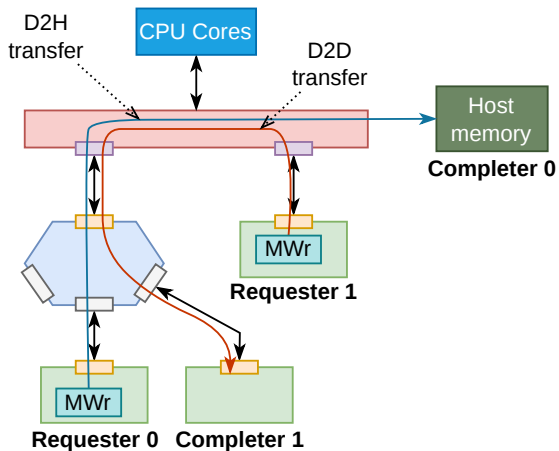


Fig.: Write request path

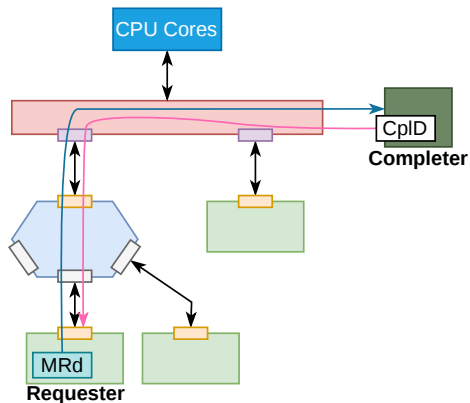
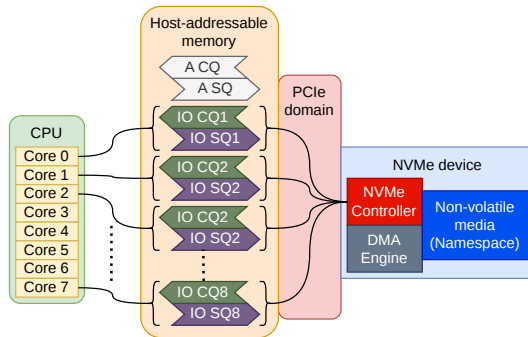
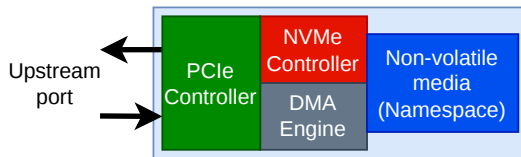


Fig.: Read request path

- Queue-pair model: submission and completion queues
- Commands are fetched by controller over PCIe
- Pointers synchronize queue state (doorbells)
- Same protocol used by host and FPGA requesters

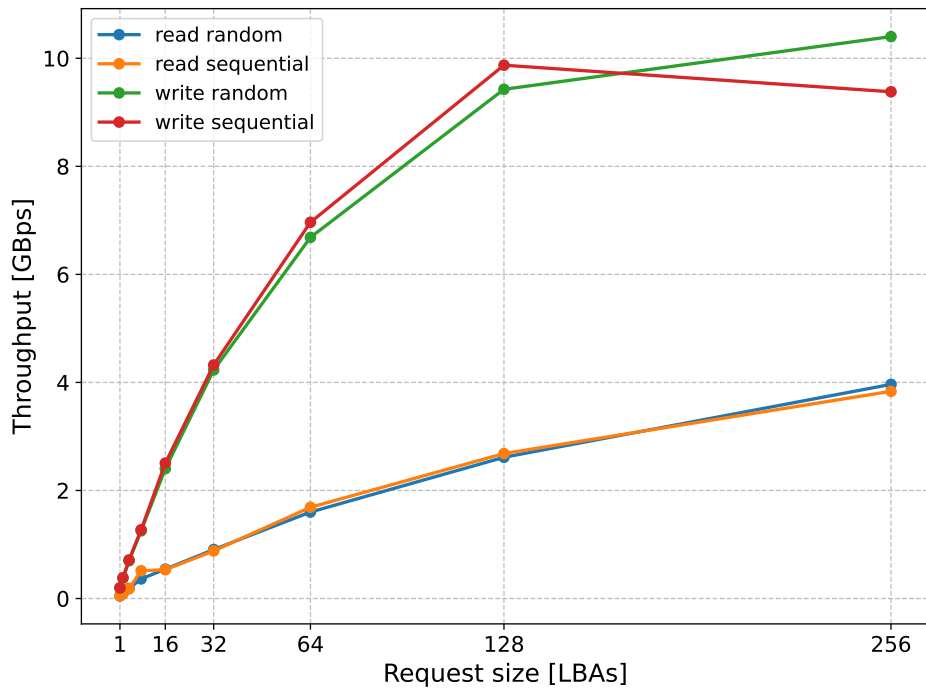


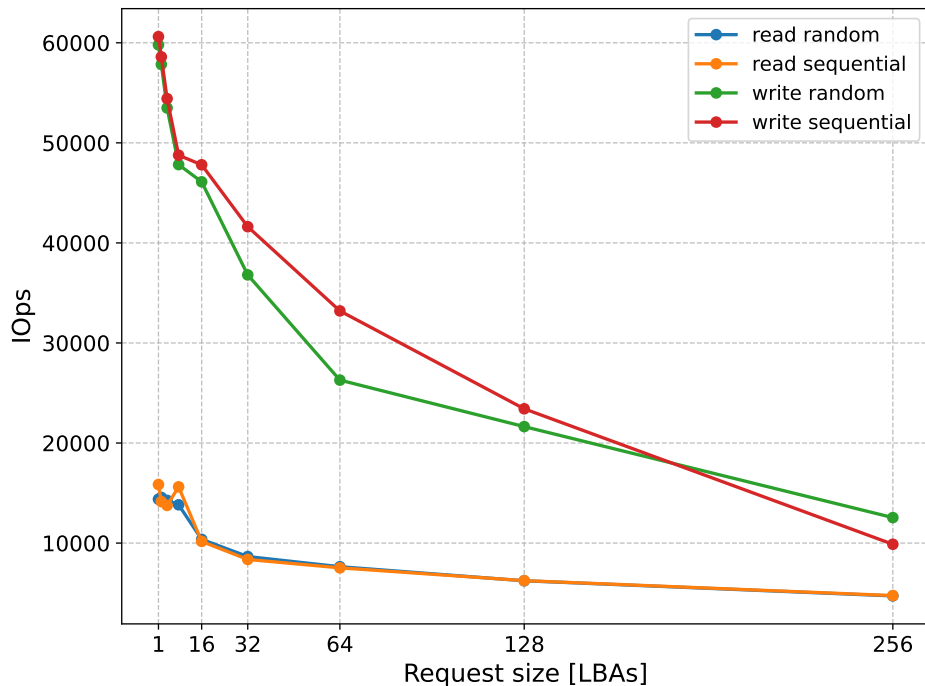
- FPGA card: AMD Alveo U55C
- Core component: DMA Iuventus IP
- Accelerator BARs hold queues and data buffers
- SPDK configures NVMe and queue resources
- Linux ublk exposes mountable block device
- Shared CPU-FPGA access to one filesystem

Table: FPGA resource consumption after design implementation phase (fully placed and routed)

Resource type	DMA luventus		PCIe IP		Test design total	
	abs	%	abs	%	abs	%
LUT	12532	0.96	9155	0.70	24792	1.90
LUTRAM	268	0.04	1148	0.19	1938	0.32
FF	22208	0.85	24903	0.96	52498	2.01
BRAM Tile	134.5	6.67	30.5	1.51	194.5	9.65
URAM	0	0	0	0	11	1.15

- Measured over period of 2^{28} cycles (~ 1 second)





- Timestamp width 28 bits (~1 second delay)
- Results stored in histogram (1 bin = 512 ns)
- 50000 repetitions

Op.	Access	Size [LBAs]	Min	P_{50}	P_{80}	P_{99}	Max
RD	Random	1	46.64	66.71	78.65	81.75	1952.72
		256	143.27	203.46	220.98	238.26	357.50
	Sequential	1	62.12	63.15	64.83	65.97	173.30
		256	144.12	201.50	218.77	237.61	1904.40
	Random	1	8.34	8.61	9.07	107.28	5379.27
		256	54.05	62.65	69.77	454.14	6692.82
WR	Sequential	1	8.34	8.45	8.60	8.70	10.19
		256	54.10	71.57	90.12	251.26	5860.76
	Random	1	8.34	8.61	9.07	107.28	5379.27
		256	54.05	62.65	69.77	454.14	6692.82

Next step: live demo of end-to-end shared read/write access.

- Design verified using functional verification (CocoTB)
- Validated in hardware
- Shared CPU-FPGA filesystem access over NVMe was demonstrated
- PCIe peer-to-peer path avoids mandatory host-memory staging
- Linux filesystem access is maintained
- Future development
 - More outstanding commands
 - Support for more NVMe drives
 - Alveo V80 support
 - Support for both NVMe and host memory transfers

Thank you for your attention!

Any questions?